8051 Stack Operations

1 Stack Overview

What is a Stack?

The **stack** is a Last-In-First-Out (LIFO) data structure used for temporary storage of data, return addresses during subroutine calls, and saving register contents during interrupts.

1.1 Stack Characteristics in 8051

- Stack operations are **8-bit wide** (1 byte at a time).
- **PUSH:** 1 byte of data is stored onto the stack.
- **POP:** 1 byte of data is retrieved from the stack.
- Stack resides in **internal RAM** (00H – 7FH).

16-bit Address Push/Pop

For **internal 16-bit address push or pop** (e.g., during CALL instruction):

- Operation is implemented **byte by byte**.
- **Lower byte first**, followed by **higher byte**.

This ensures proper restoration of the 16-bit Program Counter (PC) during subroutine returns.

2 Stack Pointer (SP) Register

Stack Pointer

The **Stack Pointer (SP)** is an **8-bit register** that points to the **top of the stack** in internal RAM.

Key Characteristics:

- **Size:** 8 bits wide.
- **Location:** SFR address `81H`.
- **Initial Value:** Initialized to **07H** after reset.
- **Stack Top:** Always assumed to be **preoccupied** (contains valid data).

Stack Top Assumption

The stack top address pointed by SP is always assumed to be **preoccupied** with data. This is why:

- **PUSH** increments SP **first**, then stores data.
- **POP** reads data **first**, then decrements SP.

Default Stack Location

After reset, SP = 07H means:

- First PUSH will store data at address **08H** (Register Bank 1 area).
- If using multiple register banks, initialize SP to a higher address (e.g., 2FH or 7FH) to avoid conflicts.

3 PUSH Operation

PUSH Instruction

The **PUSH** instruction stores data onto the stack. It follows a specific sequence to ensure proper stack management.

3.1 PUSH Operation Steps

PUSH Sequence

Step 1: Increment Stack Pointer (SP) by 1.

$$\text{SP} \leftarrow \text{SP} + 1$$

Step 2: Store the 8-bit content of the address specified in the instruction to the memory location pointed by SP.

$$(\text{SP}) \leftarrow \text{Data from specified address}$$

3.2 PUSH Example

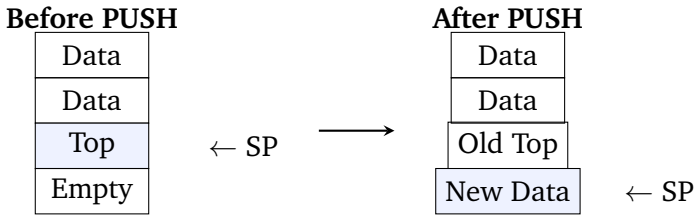
PUSH Code Example

```
; Assume SP = 07H, ACC = 55H

PUSH ACC        ; Push accumulator onto stack

; After execution:
; SP = 08H
; Memory[08H] = 55H
```

Execution Flow:

1. SP incremented: $07\text{H} \rightarrow 08\text{H}$
2. Content of ACC (55H) stored at address 08H

3.3 PUSH Diagram



4 POP Operation

POP Instruction

The **POP** instruction retrieves data from the stack. It follows a specific sequence that is the reverse of PUSH.

4.1 POP Operation Steps

POP Sequence

Step 1: Store the content of the top of stack (pointed by SP) to the 8-bit memory address specified in the instruction.

$$\text{Specified address} \leftarrow (\text{SP})$$

Step 2: Decrement Stack Pointer (SP) by 1.

$$\text{SP} \leftarrow \text{SP} - 1$$

4.2 POP Example

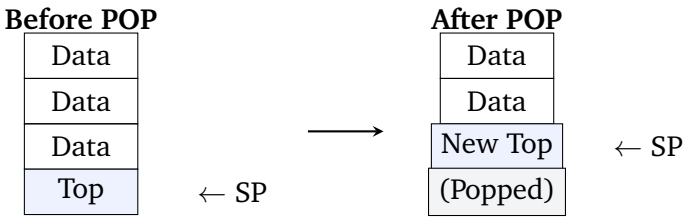
POP Code Example

```
; Assume SP = 08H, Memory[08H] = 55H

POP ACC          ; Pop top of stack into accumulator

; After execution:
; ACC = 55H
; SP = 07H
```

Execution Flow:

1. Content at address 08H (55H) copied to ACC
2. SP decremented: $08\text{H} \rightarrow 07\text{H}$

4.3 POP Diagram



5 PUSH vs POP Comparison

Aspect	PUSH	POP
Order	SP increment first, then store	Read first, then SP decrement
SP Change	SP $\leftarrow$ SP + 1	SP $\leftarrow$ SP − 1
Data Flow	Register/Memory $\rightarrow$ Stack	Stack $\rightarrow$ Register/Memory
Stack Growth	Stack grows upward (higher addresses)	Stack shrinks downward

6 Summary: Stack Operations

Quick Reference Summary

Stack Pointer (SP):

- 8-bit register at SFR address 81H.
- Initialized to 07H after reset.
- Stack top is always assumed to be preoccupied.

PUSH Operation:

1. Increment SP by 1.
2. Store data at new SP address.

POP Operation:

1. Read data from current SP address.
2. Decrement SP by 1.

16-bit Operations:

- Implemented byte by byte.
- Lower byte first, then higher byte.

The stack is essential for subroutine calls, interrupt handling, and temporary data storage in 8051 programming.